

MEMÓRIA DESCRITIVA

Portal de Dados: Data4Moz

Documento técnico elaborado com base na análise do código-fonte e da arquitectura implementada.

Versão do projecto: 1.0.0 (package.json) · Stack: Next.js 14 Full Stack · Base de dados: MySQL

1. Serviços Prestados

1.1. Objecto e finalidade

O Portal de Dados (DataPortal) é uma plataforma web aberta desenvolvida pela Data4Moz que centraliza a publicação, consulta e descarregamento de datasets, dashboards alfanuméricos, mapas inteligentes com painéis analíticos, relatórios e mecanismos de contacto. A aplicação destina-se a disponibilizar informação territorial e estatística de forma acessível a instituições, academia, empresas e sociedade civil.

1.2. Tipos de utilizadores

- Visitantes (público): acedem sem autenticação a catálogos, mapas, dashboards, relatórios, pesquisa e formulários de contacto/pedidos.
- Administrador: único perfil autenticado existente no sistema. Não há registo nem contas de utilizador comum. O administrador gere conteúdos via painel em /admin e consulta estatísticas em /dashboard.

1.3. Serviços e funcionalidades implementadas

1.3.1. Página inicial

- Apresentação institucional (Hero, Sobre, Funcionalidades, FAQ, parceiros).
- Estatísticas agregadas da base de dados: total de datasets, visualizações, downloads e número de fontes/organizações (app/page.tsx).
- Catálogo em destaque com datasets mais consultados.

- Pesquisa unificada com sugestões (SearchSuggestionsPopover + API /api/search/suggestions).

1.3.2. Catálogo de datasets

- /dados-espaciais — catálogo geoespacial com filtros, pesquisa, pré-visualização em mapa (Leaflet) e paginação infinita (components/geo/).
- /dados-alfanumericos — catálogo alfanumérico com filtros e listagem (components/alf/).
- /catalogo — catálogo unificado de datasets.
- /dataset/[id] — página de detalhe com metadados, pré-visualização de ficheiros (CSV, Excel, GeoJSON, shapefile ZIP via shpjs), checksum SHA-256 e descarregamento.
- Tipos de dados suportados na base de dados: dataType = geoespacial | alfanumerico (lib/db.ts).

1.3.3. Descarregamento e estatísticas de utilização

- GET /api/download/[id] — entrega o ficheiro armazenado em public/uploads/ e incrementa contadores de downloads.
- Registo de visualizações e downloads por dataset (campos views/downloads e tabela Statistic).
- GET /api/datasets/[id]/preview — pré-visualização tabular ou geográfica.
- GET /api/datasets/[id]/checksum — hash SHA-256 do ficheiro para verificação de integridade.

1.3.4. Dashboards alfanuméricos

- /dashboards-alfanumericos — galeria de painéis externos (Power BI, ArcGIS, etc.) registados na tabela AlphanumericDashboard.
- Integração por URL de embed (lib/dashboard-utils.ts) com pré-visualização em iframe ou imagem.
- Contador de visualizações por clique em «Ver mais» (POST /api/alphanumeric-dashboards/[id]/view).

1.3.5. Mapas inteligentes (mapas + dashboards geoespaciais)

- /maps — catálogo estático de experiências publicadas (lib/maps-catalog.ts).

- Quatro mapas implementados, cada um com componente React dedicado e ficheiros de dados em public/data/:
- • mapa-de-saude — Mapa Inteligente de Saúde Pública (health-adm3.geojson).
- • diagnostico-rede-postes — Análise e Diagnóstico da Infraestrutura Eléctrica (poles-network.json).
- • malaria-geografia-2015-2018 — Série Temporal e Geográfica de Incidência de Malária (malaria-provinces.json).
- • feederpulse-mz — FeederPulse-MZ, inteligência energética (feeder-pulse.json).
- Visualização com Leaflet, gráficos Recharts/Chart.js, KPIs e filtros conforme cada dashboard.

1.3.6. Relatórios

- /relatorios e /relatorios/[id] — listagem e detalhe de relatórios (tabela Report).
- Pedido de acesso/informação via ReportRequestButton → POST /api/report-requests (registo na tabela ReportRequest).

1.3.7. Contacto e comunicação

- Modal de contacto global (ContactModalProvider, ContactFloatingButton).
- POST /api/contact — grava mensagem na tabela ContactMessage; envio opcional por e-mail via SMTP (Nodemailer, lib/mailer.ts).
- Rate limiting: 5 pedidos/hora por IP para contacto.

1.3.8. Páginas institucionais e legal

- /termos-condicoes e /politica-cookies — termos e política de cookies.
- TermsConsentModal — registo de consentimento em localStorage (dataPortalTermsConsent).
- /ai-insights — página informativa/demo sobre capacidades de IA (sem backend de IA implementado).

1.3.9. Administração (requer autenticação)

- /admin/login — autenticação do administrador.
- /admin — painel CRUD: datasets, categorias, relatórios, dashboards alfanuméricos (AdminPanel).

- /dashboard — painel analítico com gráficos de utilização (ModernDashboard).
- Upload de ficheiros via POST /api/upload (máx. 100 MB, autenticação obrigatória).
- Exportação de relatórios administrativos: JSON, CSV (GET /api/admin/reports) e PDF (GET /api/admin/reports/pdf, jsPDF).
- Analytics: GET /api/admin/analytics — top downloads/visualizações e série temporal.

1.4. Funcionalidades não implementadas (transparência técnica)

- Registo ou autenticação de utilizadores finais (apenas administrador).
- Módulo AI Insights operacional (apenas página promocional).
- NextAuth — variável NEXTAUTH_URL documentada em .env.example mas não utilizada; autenticação é JWT customizada.
- Scripts automatizados de backup da base de dados ou ficheiros (não existem no repositório).

2. Aspectos Técnicos da Arquitectura e dos Sistemas de Informação

2.1. Stack tecnológica

Camada	Tecnologia	Evidência
Runtime / Framework	Node.js, Next.js 14.2 (App Router)	package.json, app/
Front-end	React 18, TypeScript, Tailwind CSS 4	components/, tailwind.config.ts
Back-end	Next.js Route Handlers (app/api/)	23 rotas API
Base de dados	MySQL (mysql2/promise, pool)	lib/db.ts, DATABASE_URL
Autenticação	JWT (jsonwebtoken) + cookie httpOnly	lib/auth.ts
Palavras-passe	bcryptjs (cost factor 10)	app/api/auth/login, scripts/create-admin.ts
Mapas	Leaflet, react-leaflet	components/maps/, components/geo/
Gráficos	Recharts, Chart.js, react-chartjs-2	components/maps/, ModernDashboard

Relatórios PDF	jsPDF, jspdf-autotable	app/api/admin/reports/pdf/route.ts
E-mail	Nodemailer (SMTP)	lib/mailer.ts
Ficheiros geo	shpjs (shapefile), xlsx (Excel)	app/api/datasets/[id]/preview/route.ts

2.2. Arquitectura geral

O sistema adopta uma arquitectura monolítica full-stack em Next.js: páginas React (Server e Client Components) coexistem com Route Handlers REST na mesma aplicação. A lógica de persistência concentra-se em lib/db.ts com SQL parametrizado. Não existe camada ORM activa em runtime (existem migrações SQL em prisma/migrations/ apenas como referência de schema).

2.3. Comunicação Front-end ↔ Back-end

- Páginas server-side acedem directamente a lib/db.ts (ex.: app/page.tsx, catálogos).
- Componentes client-side invocam fetch() às rotas /api/* (login, CRUD admin, contacto, pesquisa, preview, download).
- Server Actions em app/dataset/[id]/actions.ts para registo de visualizações.
- Autenticação admin: POST /api/auth/login define cookie auth-token; pedidos autenticados leem cookie no servidor via getCurrentUser().

2.4. Comunicação com MySQL

- Pool de ligações: connectionLimit 10, uri = process.env.DATABASE_URL (lib/db.ts).
- Todas as queries utilizam placeholders ? (prepared statements) — protecção contra SQL injection.
- Cache em memória de 60 segundos para listagens de datasets (datasetsQueryCache).
- Migrações runtime: ensureCategoryCompositeUnique(), ensureAlphanumericDashboardTable().

2.5. Organização de pastas

app/ – Rotas, layouts, API (App Router)

components/ – UI React (geo, alf, maps, dashboards, admin, ...)

lib/ – db, auth, security, mailer, maps-catalog, portal-search

public/ – Imagens, uploads/, data/ (JSON/GeoJSON estáticos)

prisma/migrations/ – SQL de schema inicial

scripts/ – create-admin, seed, extracção de dados

middleware.ts – Headers de segurança e CORS

next.config.js – CSP, HSTS, external packages

2.6. Modelo de dados (tabelas)

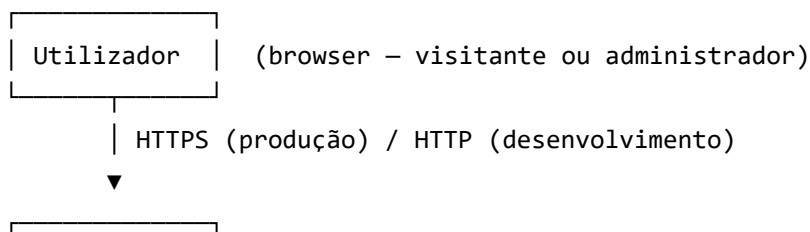
- User — administradores (email único, password hash).
- Category — categorias com dataType (geoespacial, alfanumerico, dashboard).
- Dataset — metadados e filePath para ficheiros em public/.
- Statistic — eventos view/download por dataset.
- Report — relatórios publicados.
- ReportRequest — pedidos de relatório (criada em runtime, sem migração SQL no repo).
- ContactMessage — mensagens do formulário de contacto.
- AlphanumericDashboard — dashboards externos embedáveis.

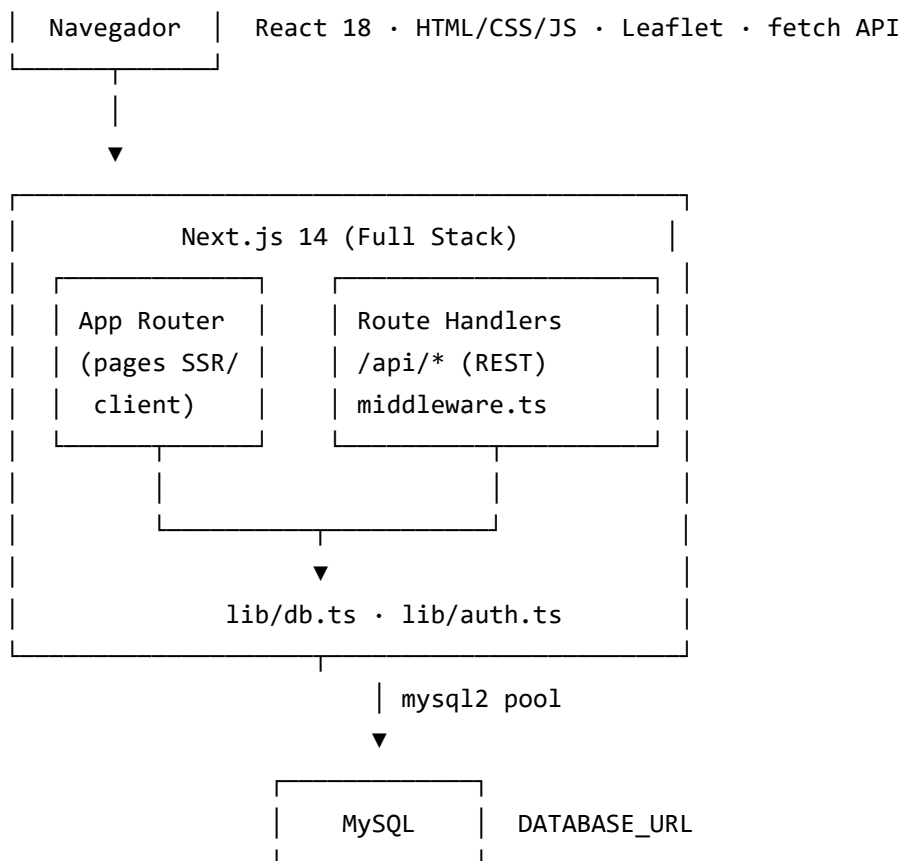
3. Descrição e Diagramas da Infraestrutura Tecnológica

3.1. Componentes de infraestrutura

A aplicação é uma aplicação web Node.js que pode ser executada em modo desenvolvimento (next dev), produção (next start) ou via server.js (servidor HTTP customizado com PORT configurável).

3.2. Diagrama de fluxo (ASCII)





Armazenamento de ficheiros: `public/uploads/` (datasets/relatórios)

Dados estáticos de mapas: `public/data/*.json|geojson`

3.3. Serviços externos

- MySQL — base de dados relacional (obrigatório).
- SMTP — envio de e-mails do formulário de contacto (opcional; `CONTACT_RECIPIENT_EMAIL`, `SMTP_*`).
- OpenStreetMap / OpenTopoMap / ArcGIS Online — tiles de mapas (client-side, CSP allowlist).
- Power BI / ArcGIS — embeds de dashboards alfanuméricos (frame-src na CSP).
- unpkg.com — permitido na CSP para imagens (configuração `next.config.js`).

3.4. Alojamento e armazenamento

- Aplicação: qualquer ambiente Node.js capaz de executar Next.js 14 (VPS, cloud, PaaS — ex.: Vercel referenciado em `lib/site.ts` via `VERCEL_URL`).

- URL de produção configurável: NEXT_PUBLIC_SITE_URL (fallback documentado: <https://dataportal.co.mz>).
- Dados estruturados: servidor MySQL acessível via DATABASE_URL.
- Ficheiros binários: sistema de ficheiros local em public/uploads/ no servidor da aplicação.
- Dados estáticos de mapas analíticos: public/data/ incluídos no deploy.

4. Aspectos de Segurança dos Sistemas de Informação

4.1. Autenticação e controlo de acesso

- Login exclusivo de administrador: POST /api/auth/login com email + password.
- JWT assinado com JWT_SECRET, validade 7 dias, payload { userId, email }.
- Cookie auth-token: httpOnly, secure em produção, sameSite strict (prod) / lax (dev), path=/.
- Em produção, recusa arranque/verificação se JWT_SECRET mantiver valor por defeito inseguro (lib/auth.ts).
- Rotas /admin e /dashboard protegidas server-side com getCurrentUser() + redirect para /admin/login.
- Operações de escrita (POST/PUT/DELETE) nas APIs verificam getCurrentUser() individualmente.
- Nota: middleware.ts não bloqueia /admin; a protecção é feita nas páginas e APIs, não na middleware.

4.2. Palavras-passe

- Armazenamento com bcrypt (bcryptjs), factor de custo 10.
- Script create-admin exige password forte: ≥ 12 caracteres, maiúsculas, minúsculas, dígito e símbolo (isStrongPassword em lib/security.ts).

4.3. Protecção de inputs e abusos

- SQL injection: queries parametrizadas em lib/db.ts e rotas API.
- Validação: normalizeText (limite de comprimento), normalizeEmail, isValidEmail.

- Rate limiting em memória (lib/security.ts): login 10/15 min/IP; contacto 5/h/IP; report-requests 20/h/IP.
- Upload: requer autenticação; limite 100 MB; nomes únicos com timestamp — sem whitelist de MIME/extensão documentada.

4.4. Headers HTTP e políticas

- middleware.ts: X-Frame-Options DENY, X-Content-Type-Options nosniff, Referrer-Policy, Permissions-Policy.
- next.config.js: Strict-Transport-Security (HSTS), Content-Security-Policy restrictiva.
- CORS configurável via CORS_ALLOWED_ORIGINS para rotas /api/*.
- poweredByHeader: false (oculta X-Powered-By).

4.5. XSS e exposição

- React escapa conteúdo por defeito; não foi identificada biblioteca de sanitização HTML adicional.
- robots.ts: disallow /admin, /dashboard, /api/ para indexação.

4.6. HTTPS

- HSTS configurado globalmente; cookie secure activo em NODE_ENV=production.
- Implementação efectiva de HTTPS depende do reverse proxy / alojamento em produção.

5. Aspectos de Protecção de Dados

5.1. Armazenamento

- Metadados de catálogo, utilizadores admin, mensagens de contacto e pedidos: MySQL (utf8mb4).
- Ficheiros de datasets e relatórios: disco local public/uploads/, referenciados por filePath na base de dados.
- Dados analíticos de mapas publicados: ficheiros estáticos em public/data/.
- Consentimento de termos: localStorage no browser (TermsConsentModal).

5.2. Quem acede aos dados

- Público: consulta e descarregamento de datasets/relatórios publicados; submissão de contacto e pedidos de relatório.
- Administrador autenticado: CRUD completo, upload, analytics e exportações administrativas.
- Base de dados e uploads: acesso restrito à infraestrutura de alojamento e credenciais DATABASE_URL / servidor.

5.3. Confidencialidade, integridade e disponibilidade

- Confidencialidade: passwords hash bcrypt; JWT em cookie httpOnly; APIs admin autenticadas; segredos em variáveis de ambiente (.env, não commitados).
- Integridade: checksum SHA-256 disponível por dataset (GET /api/datasets/[id]/checksum); validação de inputs; SQL parametrizado.
- Disponibilidade: pool MySQL com limite de ligações; cache de leitura de datasets (60s); dependência de disponibilidade do servidor MySQL e do host da aplicação.

5.4. Backup

O repositório não inclui scripts automatizados de backup da base de dados nem dos ficheiros em public/uploads/. A política de backup deve ser definida ao nível da infraestrutura de alojamento (snapshots MySQL, cópias periódicas do filesystem).

5.5. Boas práticas adoptadas no projecto

- Segredos externalizados (.env.example como modelo, sem credenciais reais).
- Princípio de mínimo privilégio na API: operações sensíveis exigem sessão admin.
- Limitação de taxa em endpoints públicos expostos a abuso (login, contacto, pedidos).
- Política de cookies e termos publicadas; modal de consentimento na primeira visita.
- Metadados descritivos por dataset (fonte, cobertura, ano, keywords) para reutilização responsável.

6. Anexos técnicos

6.1. Rotas API (inventário)

POST	/api/auth/login	/api/auth/logout
GET/POST	/api/categories	PUT/DELETE /api/categories/[id]
GET/POST	/api/datasets	PUT/DELETE /api/datasets/[id]
GET	/api/datasets/count	/api/datasets/[id]/preview
GET	/api/datasets/[id]/checksum	/api/download/[id]
POST	/api/upload	/api/contact
GET/POST	/api/reports	PUT/DELETE /api/reports/[id]
POST	/api/report-requests	
GET/POST	/api/alphanumeric-dashboards	PUT/DELETE /api/alphanumeric-dashboards/[id]
POST	/api/alphanumeric-dashboards/[id]/view	
GET	/api/dashboard-categories	/api/search/suggestions
GET	/api/admin/analytics	/api/admin/reports
	/api/admin/reports/pdf	

6.2. Variáveis de ambiente (.env.example)

DATABASE_URL, JWT_SECRET, CONTACT_RECIPIENT_EMAIL
 SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, SMTP_SECURE
 CORS_ALLOWED_ORIGINS, NEXTAUTH_URL (não usado pela auth actual)
 Adicionalmente no código: NEXT_PUBLIC_SITE_URL,
 NEXT_PUBLIC_PORTAL_EMAIL, VERCEL_URL, NODE_ENV, PORT

— *Fim da Memória Descritiva* —

Elaborado com base na análise do código-fonte do projecto DataPortal (Data4Moz).